CHECKPOINT 5

1. ¿QUÉ ES UN CONDICIONAL?

Un condicional en Python es una manera de hacer que una aplicación sea dinámica, como si tomara decisiones. Ejecutará ciertas líneas de código solo si se cumplen la o las condiciones necesarias.

CONDICIONAL BÁSICO

Usaremos **if** y **else**.

En el siguiente ejemplo indicamos que si la edad de una persona es menor que 15, aparecerá un texto concreto. En el ejemplo hemos usado f para poder incluir la variable aunque en este caso no la hemos incluido, así que podríamos no ponerlo.

```
1 edad = 15
2
3 if edad < 25:
4     print(f"Lo siento, debes tener por lo menos 25 años")
5
```

Salida

```
Lo siento, debes tener por lo menos 25 años
```

Si no se cumple la condición no pone nada porque no hemos configurado el “falso”.

Añadimos el “falso”:

```
1 edad = 35
2
3 if edad < 25:
4     print(f"Lo siento, debes tener por lo menos 25 años.")
5 else:
6     print(f"Bien, {edad} está en el rango de edad para alquilar un coche.")
```

Salida

```
Bien, 35 está en el rango de edad para alquilar un coche.
```

Si queremos añadir más condiciones usaremos **elif** (es como else if):

```
1 edad = 125
2
3 if edad < 25:
4     print(f"Lo siento, debes tener por lo menos 25 años.")
5 elif edad > 100:
6     print(f"Lo siento, {edad} años son demasiados.")
7 else:
8     print(f"Bien, {edad} está en el rango de edad para alquilar un coche.")
```

Salida

```
Lo siento, 125 años son demasiados.
```

OPERADOR TERNARIO

Se llama operador ternario a una manera más corta de escribir los condicionales. Es interesante si el código a meter ocupa una línea, si es más ya se está complicado más de la cuenta, y una de las buenas prácticas de Python es la sencillez. Si es más fácil con el código normal no deberíamos usar el operador ternario.

```python
rol = "admin"

autorizacion = "Acceso permitido" if rol == "admin" else "Acceso denegado"

print(autorizacion)
```

Entrada del programa

Salida

```
Acceso permitido
```

En este ejemplo vemos que con el operador ternario escribimos casi como hablamos. Este es un buen ejemplo porque es sencillo, no ocupa más de una línea.

Si lo hiciéramos de la forma habitual sería:

```python
rol = "invitado"

if rol == "admin":
    autorizacion = "Acceso permitido"
else:
    autorizacion = "Acceso denegado"

print(autorizacion)
```

Entrada del programa

Salida

```
Acceso denegado

[Execution complete w
```

LISTA COMPLETA DE OPERADORES CONDICIONALES EN PYTHON

- == Igual
- != Distinto
- <> Distinto (obsoleto)
- >, >=, <, <= Como en Excel: estos los usamos con números.

```python
username = "Margari"

if username != "Margari":
    print("Bienvenida a Ateinru")
else:
    print("¡No puedes pasar!")
```

Entrada del programa

Salida

```
¡No puedes pasar!
```

```python
username = "Margari"
edad = 25

if edad >=20:
    print("Bienvenida a Ateinru")
else:
    print("¡No puedes pasar!")
```

Entrada del programa

Salida

```
Bienvenida a Ateinru
```

Podemos comparar también listas:

```python
lista_uno = ["Margari", "Mirari"]
lista_dos = ["Begochu", "Patxi"]

if lista_uno == lista_dos:
    print("Son iguales")
else:
    print("Son distintas")
```

Entrada del programa

Salida

```
Son distintas
```

Lo podemos complicar un poco más:

```python
lista_uno = ["Margari", "Mirari"]
lista_dos = ["Begochu", "Margari"]

if lista_uno[0] == lista_dos[1]:
    print("Son iguales")
else:
    print("Son distintas")
```

Entrada del program

Salida

```
Son iguales
```

CONDICIONALES COMPUESTOS (OPERADORES AND, AND NOT Y OR)

En ocasiones tenemos condiciones que no son únicas sino que se deben cumplir varias a la vez (**and**) o alguna de varias (**or**), o excluyendo alguna (**and not**).

Nos permite afinar mucho más en lo que queremos y lo que no.

En el siguiente ejemplo se deben cumplir dos condiciones a la vez, usamos **and**:

```python
username ="japiyou"
correo = "japiyou@gmail.com"
contraseña = "ateinru"

if username == "japiyou" and contraseña =="ateinru":
    print("Acceso permitido")
else:
    print("Acceso denegado")
```

Entrada del programa

Salida

```
Acceso permitido

[Execution complete with exit code 0]
```

En el siguiente ejemplo se debe cumplir alguna de las dos condiciones, usamos **or**:

```python
username ="japiyou"
correo = "japiyou@gmail.com"
contraseña = "ateinru"

if username == "japiyou" or contraseña =="ate":
    print("Acceso permitido")
else:
    print("Acceso denegado")
```

Entrada del programa

Salida

```
Acceso permitido

[Execution complete with exit code 0]
```

Podemos combinar and y or usando paréntesis:

```python
username ="pepi"
correo = "japiyou@gmail.com"
contraseña = "ateinru"

if (username == "japiyou" and contraseña =="ateinru") or correo == "japiyou@gmail.com":
    print("Acceso permitido")
else:
    print("Acceso denegado")
```

Entrada del programa

Salida

```
Acceso permitido

[Execution complete with exit code 0]
```

```python
username ="pepi"
correo = "japi@gmail.com"
contraseña = "ateinru"

if (username == "japiyou" and contraseña =="ateinru") or correo == "japiyou@gmail.com":
    print("Acceso permitido")
else:
    print("Acceso denegado")
```

Entrada del programa

Salida

```
Acceso denegado

[Execution complete with exit code 0]
```

Otro ejemplo, esta vez con True o False, usamos **and not** para lo que queremos que no se cumpla.

```python
logeado = True
usuario_estandar = True

if logeado and not usuario_estandar:
    print("Tienes acceso al panel de administrador")
else:
    print("Tienes acceso al panel de usuario estándar")
```

Entrada del programa

Salida

```
Tienes acceso al panel de usuario estándar

[Execution complete with exit code 0]
```

```python
logeado = True
usuario_estandar = False

if logeado and not usuario_estandar:
    print("Tienes acceso al panel de administrador")
else:
    print("Tienes acceso al panel de usuario estándar")
```

Entrada del programa

Salida

```
Tienes acceso al panel de administrador
```

2. ¿CUÁLES SON LOS DIFERENTES TIPOS DE BUCLES EN PYTHON? ¿POR QUÉ SON ÚTILES?

Tenemos dos tipos de bucles o loops:

- **for...in:** con este tipo de bucles se itera el número de veces que haya elementos. No nos hace falta saber el número de items que hay en la colección, ya sea lista, tupla o diccionario porque simplemente cuando no haya más parará.
- **while:** es menos inteligente que el anterior. Deberemos decirle cuándo parar y si no lo hacemos se convertirá en un bucle infinito.

El 95% de las veces usaremos un bucle for...in porque es mejor, pero hay algunos casos en los que el loop while funciona mejor, los veremos más adelante.

IMPLEMENTAR LOOPS EN LISTAS, TUPLES Y DICCIONARIOS

Veremos 3 tipos de estructuras clave para usar loops.

Es muy importante que después del for...in se use sangría, tenemos que comprobar que se pone sola y si no ponerla.

LOOP EN LISTAS

Usamos la siguiente estructura:

```
1 amigas = ["Margari", "Mirari", "Begochu", "Marisa"]
2
3 for amiga in amigas:
4     print(amiga)
```

Entrada del programa

Salida

```
Margari
Mirari
Begochu
Marisa
```

Explicación:

- amigas es el nombre de la variable, este tiene que ser este nombre.
- amiga, en realidad, podría ser cualquier nombre, pero la convención suele ser usar el singular de la variable. Se puede llamar variable de bloque o variable iteradora.

LOOP EN TUPLAS

Funcionará de la misma manera:

```
1 amigas = ("Margari", "Mirari", "Begochu", "Marisa")
2
3
4 for amiga in amigas:
5     print(amiga)
```

Entrada del progr

Salida

```
Margari
Mirari
Begochu
Marisa
```

LOOP EN DICCIONARIOS

Si uso la misma estructura que antes no me da toda la información, porque los elementos del diccionario tienen claves:

```python
familiares = {
    "marido": "Fer",
    "hija1": "Claudi",
    "hija2": "Juni",
    "perro": "Kofi"
}

for familiar in familiares:
    print(familiar)
```

Entrada del pr

Salida

```
marido
hija1
hija2
perro
```

Entonces, en un diccionario tenemos que hacer lo siguiente:

```python
for parentesco, familiar in familiares.items():
    print("Parentesco", parentesco)
    print("Familiar", familiar)
```

Ponemos las dos variables de bloque que hemos elegido, parentesco y familiar, y luego a la hora de imprimirlo lo añadimos como string para que aparezca también.

Salida

```
Parentesco marido
Familiar Fer
Parentesco hija1
Familiar Claudi
Parentesco hija2
Familiar Juni
Parentesco perro
Familiar Kofi
```

LOOP A TRAVÉS DE LOS CARACTERES DE UN STRING

Podemos hacer un bucle a través de los caracteres de un string. Es súper sencillo, funciona como si el string fuese una colección.

```python
alfabeto = "abcdef"

for letra in alfabeto:
    print(letra)
```

Entrada de

Salida

```
a
b
c
d
e
f
```

LOOP EN RANGOS

Podemos crear el loop en el rango y vemos que parará uno antes del segundo parámetro, tenemos que tener en cuenta esto porque si queremos que llegue al 10 tendremos que darle como rango (1,11).

```
1 for num in range(1, 10):
2     print(num)
```

Entrada del

Salida

```
1
2
3
4
5
6
7
8
9
```

Si nos queremos saltar valores usaremos el tercer parámetro del rango, que indica el salto que da, en el ejemplo siguiente estamos indicando que vaya de dos en dos.

```
1 for num in range(1, 11,2):
2     print(num)
```

Entrada d

Salida

```
1
3
5
7
9
```

ALTERAR EL COMPORTAMIENTO DEL LOOP EN ALGÚN MOMENTO

Quiero crear un bucle que cuando encuentre un valor cambie su comportamiento. Tenemos dos operadores lógicos de control de flujo diferentes:

- continue
- break

CONTINUE

Con continue, el loop sigue funcionando.

```
1 nombres = [
2     "Margari",
3     "Mirari",
4     "Begochu",
5     "Marichu",
6     "Marisa"
7 ]
8
9 for nombre in nombres:
10     if nombre == "Marichu":
11         print(f"Lo siento, {nombre}, tu no vives en Urnieta")
12         continue
13     else:
14         print(f"{nombre}, tú sí vives en Urnieta")
```

Entrada del programa

Salida

```
Margari, tú sí vives en Urnieta
Mirari, tú sí vives en Urnieta
Begochu, tú sí vives en Urnieta
Lo siento, Marichu, tu no vives en Urnieta
Marisa, tú sí vives en Urnieta

[Execution complete with exit code 0]
```

BREAK

Con break, lo que queremos es que si encuentra lo que busco pare porque no me importa el resto de cosas.

Veremos que el loop va funcionando hasta que encuentra lo que busca, y entonces para.

```python
nombres = [
    "Margari",
    "Mirari",
    "Begochu",
    "Marichu",
    "Marisa"
]

for nombre in nombres:
    if nombre == "Marichu":
        print(f"{nombre} se encuentra en el índice {nombres.index(nombre)}")
        break
    print(nombre)

```

Entrada del programa

Salida

```
Margari
Mirari
Begochu
Marichu se encuentra en el índice 3

[Execution complete with exit code 0]
```

LOOP WHILE

Habrá ocasiones en las que nos sea útil utilizar un bucle while.

Con el bucle for…in, hay un principio y un final. Con el bucle while, deberemos decirle cuándo parar, si no se convertirá en un bucle infinito. Para decirle que pare usaremos lo que se llama **valor centinela**, vamos a ver algunos ejemplos:

```python
numeros = list(range(1,100))

print(numeros)
```

Entrada del programa

Salida

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36
```

Usando for…in obtendríamos esto:

```python
numeros = list(range(1,100))

for numero in numeros:
    print(numero)
```

Entrada del programa

Salida

```
1
2
3
4
5
6
7
8
9
```

Con while es más complicado y mucho menos intuitivo:

```python
numeros = list(range(1,100))

while len(numeros) > 0:
    print(numeros.pop())
```

Salida

```
99
98
97
96
95
94
93
92
```

Lo que hacemos es decir, mientras que la longitud de la lista sea mayor que 0 (la lista al principio mide 99), imprime el último número y lo quitas (es lo que hace pop). Entonces ese pop es el valor centinela, que hace que al llegar a 0 pare. El resultado es en orden descendente.

Cuando no sabemos cuándo parar el bucle es cuando usaremos while. El ejemplo anterior no se haría con while, no tendría sentido.

3. ¿QUÉ ES UNA LISTA POR COMPRENSIÓN EN PYTHON?

Una lista por comprensión (list comprehension) en Python es una forma corta y clara de crear listas automáticamente usando una sola línea de código. Se usan varios bucles en una sola línea de código.

A continuación, hacemos un ejemplo donde tenemos una lista de 1 a 10 y escribimos el código necesario para calcular el cubo de cada número de manera normal, sin usar lista de comprensión:

```python
lista_numerica = range(1,11)
numeros_al_cubo = []

for numero in lista_numerica:
    numeros_al_cubo.append(numero**3)

print(list(lista_numerica))
print(numeros_al_cubo)
```

Entrada del programa

Salida

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
```

Usando una lista de comprensión será de la siguiente manera:

```python
lista_numerica = range(1,11)

numeros_al_cubo = [numero**3 for numero in lista_numerica]

print(list(lista_numerica))
print(numeros_al_cubo)
```

Entrada del programa

Salida

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
```

Al nombrar la variable números al cubo, indico que es una lista [] y directamente pongo dentro la acción que se deberá hacer para cada uno de los elementos. La variable de iteración nos la hemos inventado, podría valer cualquiera, por ejemplo aquí lo he sustituido por x:

```python

numeros_al_cubo = [x**3 for x in lista_numerica]

print(list(lista_numerica))
print(numeros_al_cubo)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]

[Execution complete with exit code 0]
```

Hacemos un ejemplo más avanzado donde queremos agregar además una condición. Crearemos una lista que capture los números pares. Primero lo hacemos con todas las líneas de código:

```python
lista_numeros = range(1,11)

numeros_pares = []

for numero in lista_numeros:
    if numero % 2 == 0:
        numeros_pares.append(numero)

print(list(lista_numeros))
print(numeros_pares)
```

Entrada del programa

Salida

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[2, 4, 6, 8, 10]

[Execution complete with exit code 0]
```

Ahora lo hacemos con lista de comprensión. Tendremos que poner la condición if al final de la línea.

```python
lista_numeros = range(1,11)

numeros_pares = [numero for numero in lista_numeros if numero % 2 == 0]

print(list(lista_numeros))
print(numeros_pares)
```

Entrada del programa

Salida

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[2, 4, 6, 8, 10]
```

4. ¿QUÉ ES UN ARGUMENTO EN PYTHON?

Un argumento en Python es el valor que proporcionamos a una función en el momento de invocarla. La función está diseñada con sus parámetros y los argumentos son el dato que "rellenará" el lugar de dichos parámetros. Los argumentos se escriben dentro de los paréntesis de la función.

En el ejemplo siguiente, los argumentos serán Margari y Echeverría:

```
1 def nombre_completo(nombre, apellido):
2     return f"{nombre} {apellido}"
3
4 margari = nombre_completo("Margari", "Echeverría")
```

Argumentos por defecto: si la función necesita que pongamos un argumento, pero no siempre lo vamos a poner, es una buena práctica poner uno por defecto, como en el siguiente ejemplo:

```
1 def saludo(nombre = "invitado"):
2     print(f"¡Hola {nombre}!")
3
4 saludo()
5 saludo("Mirari")
```

Entrada del programa

Salida

```
¡Hola invitado!
¡Hola Mirari!
```

En caso de que invoquemos a la función saludo, pero no indiquemos ningún argumento dentro de los paréntesis, en el resultado aparecerá el argumento por defecto, en este caso invitado.

Existen varios tipos de argumentos:

1. ARGUMENTOS DETERMINADOS POR SU POSICIÓN

Al definir una función esta tiene sus parámetros. Cuando queremos usar dicha función, pasamos los argumentos y ocuparán el orden correspondiente. El primer argumento ocupará el primer lugar en la función, el segundo ocupará el segundo lugar, y así sucesivamente.

```
1 def nombre_completo(nombre, apellido):
2     print(f"{nombre} {apellido}")
3
4 nombre_completo("Begochu", "Goikoetxea")
```

Esto está bien en funciones pequeñas, pero para funciones grandes puede ser complicado. Por ello tenemos el siguiente tipo de argumentos.

2. ARGUMENTOS CON NOMBRE EN FUNCIONES DE PYTHON

La función se define igual que antes, pero a la hora de invocarla, especificamos en cada argumento cuál es su nombre. De esta manera es más fácil no equivocarse cuando tenemos funciones con varios parámetros.

```
1 def nombre_completo(nombre, apellido):
2     print(f"{nombre} {apellido}")
3
4 nombre_completo(nombre = "Begochu", apellido = "Goikoetxea")
```

Entrada del programa

Salida

```
Begochu Goikoetxea
```

Si cambiamos de orden los argumentos, como estos tienen su nombre explícito, sale igual de bien:

```
1 def nombre_completo(nombre, apellido):
2     print(f"{nombre} {apellido}")
3
4 nombre_completo(apellido = "Goikoetxea", nombre = "Begochu")
```

Entrada del programa

Salida

```
Begochu Goikoetxea
```

Descompresión en colección de argumentos: a veces tenemos una función en la que no sabemos cuántos argumentos habrá, en ese caso haremos lo siguiente:

```
1 def saludo(*args):
2     print("Hola " + " ".join(args))
3
4 saludo("Joana", "Amézaga")
5 saludo("Joana", "María", "Amézaga")
```

Entrada del programa

Salida

```
Hola Joana Amézaga
Hola Joana María Amézaga
```

Al definir la función, en lugar de los argumentos ponemos *args, esta es la convención normal en Python. El símbolo * significa que hay de descomprimir los datos.

Después une “Hola “ con args. El espacio que pone aquí + “ “.join(args) es para separar cada elemento de la tupla, podría ser otra cosa, como un guion).

Luego veo que al llamar a la función pongo los argumentos que quiera y sale bien.

Si comento la línea de print y pongo otra para imprimir args, vemos que el resultado es una tupla con los argumentos:

```
1 def saludo(*args):
2     # print("Hola " + " ".join(args))
3     print(args)
4
5 saludo("Joana", "Amézaga")
6 saludo("Joana", "María", "Amézaga")
```

Entrada del programa

Salida

```
('Joana', 'Amézaga')
('Joana', 'María', 'Amézaga')
```

Podemos trabajar no solo con este tipo de argumentos de lista descomprimida sino poner además otro:

```
1 def saludo(hora, *args):
2     print("Hola " + " ".join(args))
3
4 saludo("Mañana", "Joana", "Amézaga")
5 saludo("Tarde", "Joana", "María", "Amézaga")
```

Entrada del programa

Salida

```
Hola Joana Amézaga
Hola Joana María Amézaga
```

Aquí, el tiempo del día es un argumento normal y si lo pongo en mis argumentos, veo que sigue cogiendo mi nombre y apellidos como elementos de la tupla.

```python
def saludo(hora, *args):
    print(f"Hola {" ".join(args)}, espero que estés disfrutando la {hora}.")

saludo("Mañana", "Joana", "Amézaga")
saludo("Tarde", "Joana", "María", "Amézaga")
```

Entrada del programa

Salida

```
Hola Joana Amézaga, espero que estés disfrutando la Mañana.
Hola Joana María Amézaga, espero que estés disfrutando la Tarde.
```

3. ARGUMENTOS DE PALABRA CLAVE

Cuando se trabaja con argumentos de palabra clave, la diferencia con el desempacado de palabras clave es que entonces obteníamos una tupla y ahora un diccionario. Para usarlo hay que poner **kwargs. kwargs es una convención que se suele usar en Python pero no es que sea obligatorio, sino solo una buena práctica.

En el ejemplo vemos que al llamar a la función y no pasar argumentos, la salida de imprimirlos es un diccionario vacío.

```python
def saludo(**kwargs):
    print(kwargs)

saludo()
```

Entrada del programa

Salida

```
{}
```

Si le ponemos argumentos y los imprimimos vemos que aparecen como un diccionario, primero la palabra clave y luego el valor:

```python
def saludo(**kwargs):
    print(kwargs)

saludo(nombre = "Joana", apellido ="Amézaga")
```

Entrada del programa

Salida

```
{'nombre': 'Joana', 'apellido': 'Amézaga'}
```

Le añadimos un condicional, de manera que si estamos logueados personalice el saludo y si no estamos logueados sea un saludo genérico:

```python
def saludo(**kwargs):
    if kwargs:
        print(f"Hola {kwargs["nombre"]} {kwargs["apellido"]}, que tengas un buen día.")

    else:
        print(f"Hola invitado, que tengas un buen día.")

saludo(nombre = "Joana", apellido ="Amézaga")
```

Entrada del programa

Salida

```
Hola Joana Amézaga, que tengas un buen día.

[Execution complete with exit code 0]
```

```python
def saludo(**kwargs):
    if kwargs:
        print(f"Hola {kwargs["nombre"]} {kwargs["apellido"]}, que tengas un buen día.")

    else:
        print(f"Hola invitado, que tengas un buen día.")

saludo()
```

Entrada del programa

Salida

```
Hola invitado, que tengas un buen día.

[Execution complete with exit code 0]
```

COMBINAR TODOS LOS TIPOS DE ARGUMENTOS EN UNA FUNCIÓN DE PYTHON

Podemos usar varios tipos de argumentos a la vez, como en el siguiente ejemplo. Vamos a crear una función de saludo usando todos los tipos de argumento a la vez:

```python
def saludo(hora_del_dia, *args, **kwargs):
```

- Hora del día: argumento de posición.
- Args: argumentos desempacados, será el nombre y apellidos.
- Kwargs: argumentos de palabra clave: serán las tareas del día.

```python
def saludo(hora_del_dia, *args, **kwargs):
    print(f"Hola {" ".join(args)}, espero que estés teniendo una buena {hora_del_dia}.")

    if kwargs:
        print("Tus tareas del día son:")
        for key, val in kwargs.items():
            print(f"{key} => {val}")

saludo("mañana",
       "Joana", "Amézaga",
       primero = "Lavarte la cara",
       segundo = "Hacer la cama",
       tercero = "Estudiar")
```

Entrada del programa

Salida

```
Hola Joana Amézaga, espero que estés teniendo una buena mañana.
Tus tareas del día son:
primero => Lavarte la cara
segundo => Hacer la cama
tercero => Estudiar

[Execution complete with exit code 0]
```

Si no hay tareas, es decir, argumentos de palabra clave, el mensaje que nos arroja es más corto:

```python
def saludo(hora_del_dia, *args, **kwargs):
    print(f"Hola {" ".join(args)}, espero que estés teniendo una buena {hora_del_dia}.")

    if kwargs:
        print("Tus tareas del día son:")
        for key, val in kwargs.items():
            print(f"{key} => {val}")

saludo("mañana",
       "Joana", "Amézaga")
```

Entrada del programa

Salida

```
Hola Joana Amézaga, espero que estés teniendo una buena mañana.

[Execution complete with exit code 0]
```

5. ¿QUÉ ES UNA FUNCIÓN LAMBDA EN PYTHON?

Es una función que permite empaquetar una minifunción. Se suele usar luego dentro de funciones. Su comportamiento es bastante parecido a una variable.

La función LAMBDA se define de la siguiente manera:

```
1 nombre_completo = lambda nombre, apellido: f"{nombre} {apellido}"
2
3 print(nombre_completo("Marisa", "Loquita"))
```

Entrada del programa

Salida

```
Marisa Loquita
```

Es como cuando creamos una función pero más corto.

Ahora combinaremos la función LAMBDA con una función que crearemos:

```
1 nombre_completo = lambda nombre, apellido: f"{nombre} {apellido}"
2
3 def saludo(nombre):
4     print(f"Hola {nombre}")
5
6 saludo(nombre_completo("Marisa", "Etxebarria"))
```

Entrada del programa

Salida

```
Hola Marisa Etxebarria
```

Lambda nos permite empaquetar fácil y rápidamente una funcionalidad dentro de una variable, para luego poder usarla en otros procesos y otras partes del programa.

6. ¿QUÉ ES UN PAQUETE PIP?

Un paquete pip en Python es una biblioteca o conjunto de código reutilizable que podemos instalar fácilmente en tu entorno de Python usando la herramienta llamada pip. Con pip importaremos paquetes de la tienda Pypi (también llamada Cheeseshop).

Con pip, que es el gestor de paquetes de Python, vamos a poder:

- Instalar librerías
- Actualizarlas
- Eliminarlas

Si hemos instalado Python desde la web oficial pip viene instalado. Si no lo tenemos lo instalaremos.

En la página https://pip.pypa.io/en/stable/getting-started/ tenemos toda la información.

Descargamos y ejecutamos el archivo de esta url: https://bootstrap.pypa.io/get-pip.py

Una vez instalado comprobamos que funciona bien poniendo en el terminal pip --version

```
C:\Users\joana\Desktop>pip --version
pip 26.0.1 from C:\Users\joana\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\loca
l-packages\Python313\site-packages\pip (python 3.13)

C:\Users\joana\Desktop>
```

EJEMPLO INSTALACIÓN BIBLIOTECA NUMPY

Vamos a instalar la biblioteca Numpy. Es un paquete que permite procesar números, registros y objetos. Es una de las bibliotecas más populares de la comunidad Python.

En la web https://pypi.org/ buscamos numpy.

Vamos a instalarlo desde la terminal escribiendo: pip install numpy

```
C:\Users\joana>pip install numpy
Defaulting to user installation because normal site-packages is not writeable
Collecting numpy
  Downloading numpy-2.4.4-cp313-cp313-win_amd64.whl.metadata (6.6 kB)
Downloading numpy-2.4.4-cp313-cp313-win_amd64.whl (12.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.3/12.3 MB 54.1 MB/s  0:00:00
Installing collected packages: numpy
  WARNING: The scripts f2py.exe and numpy-config.exe are installed in 'C:\Users\joana\AppData\Local\Packages\PythonSoftw
areFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed numpy-2.4.4

C:\Users\joana>
```

En mi caso me ha dado un mensaje de alerta, porque el directorio donde se instala no está en el PATH, que es el grupo de carpetas donde Windows busca comandos. Hago un primer ejemplo usando el paquete numpy. Para usarlo lo importamos: import numpy as np (esto significa que lo importo y lo renombro como np para que sea más corto). Después se usan comandos del paquete numpy (en esta explicación no es relevante profundizar), que hacen mucho más fácil la tarea, que también podría realizarse sin numpy pero de una manera más complicada.

```
C:\Users\joana>python
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb  3 2026, 18:22:25) [MSC v.1944
 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy as np
>>> rango_numeros = np.arange(16)
>>> rango_numeros
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15])
>>> rango_numeros.reshape(4,4)
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
>>>
```